



Department of Computer Science and Engineering

10CS65: Computer Graphics and Visualization

Unit 5

Geometric Objects and Transformations - 2

By

Mrs. Srividya M S



Objectives

- How to carry out transformations in OpenGL
 - Translation
 - Rotation
 - Scaling
- Current Transformation Matrix
- Push and pop operations in CTM
- Quaternions and their advantages

2D Transformations

Transformation	Equation	Homogeneous Equation
Translation	$\begin{aligned} x' &= x + \alpha_x \\ y' &= y + \alpha_y \end{aligned}$ $\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} \alpha_x \\ \alpha_y \end{pmatrix}$	$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & \alpha_x \\ 0 & 1 & \alpha_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$
Rotation	$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$	$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$
Scaling	$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \beta_x & 0 \\ 0 & \beta_y \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$	$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \beta_x & 0 & 0 \\ 0 & \beta_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$

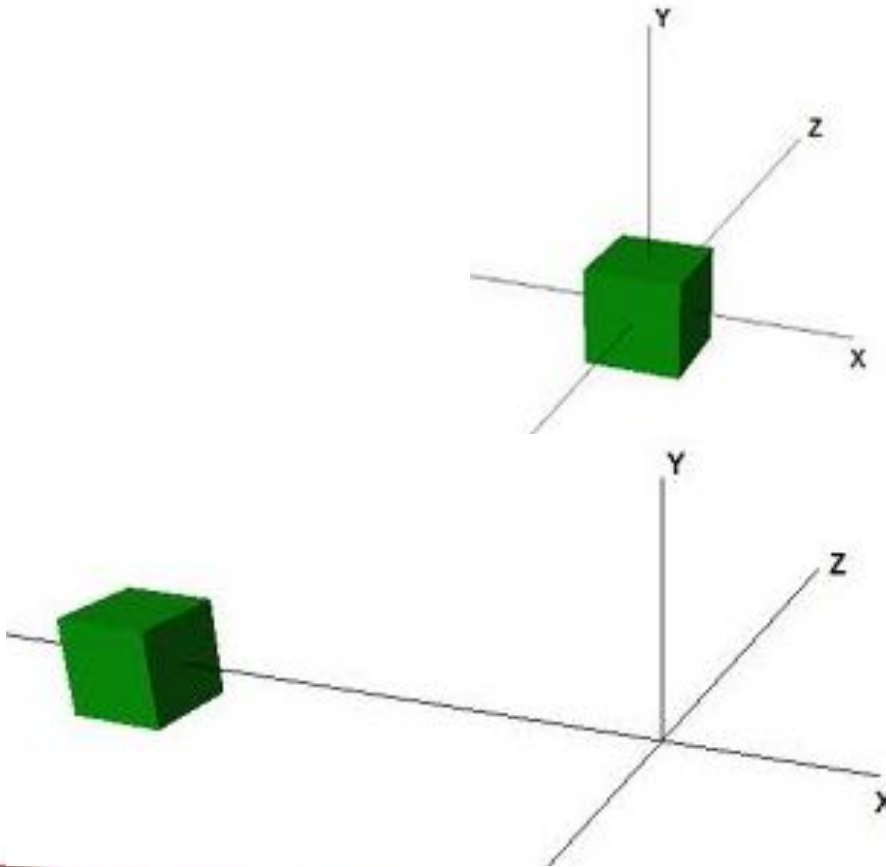
3D Transformations

Transformations in Homogeneous Coordinates

Within a frame each affine transformations can be represented as 4 * 4 matrix.

$$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Translati



TRANSLATION

- Displaces a given point to a new position $p' = p + d$
- The homogeneous co-ordinate forms of p , p' and d are

$$p' = \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} \quad p = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad d = \begin{pmatrix} \alpha_x \\ \alpha_y \\ \alpha_z \\ 0 \end{pmatrix}$$

$$x' = x + \alpha_x$$

$$y' = y + \alpha_y$$

$$z' = z + \alpha_z$$

TRANSLATION

$$x' = x + \alpha_x, y' = y + \alpha_y, z' = z + \alpha_z$$

- This involves addition of column matrix.
- This does not combine well with other affine transformations.

Matrix multiplication Form

$$p' = Tp \quad \text{where} \quad T = \begin{pmatrix} 1 & 0 & 0 & \alpha_x \\ 0 & 1 & 0 & \alpha_y \\ 0 & 0 & 1 & \alpha_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

T is known as Translation Matrix

TRANSLATION

$$p' = \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & \alpha_x \\ 0 & 1 & 0 & \alpha_y \\ 0 & 0 & 1 & \alpha_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} x + \alpha_x \\ y + \alpha_y \\ z + \alpha_z \\ 1 \end{pmatrix}$$

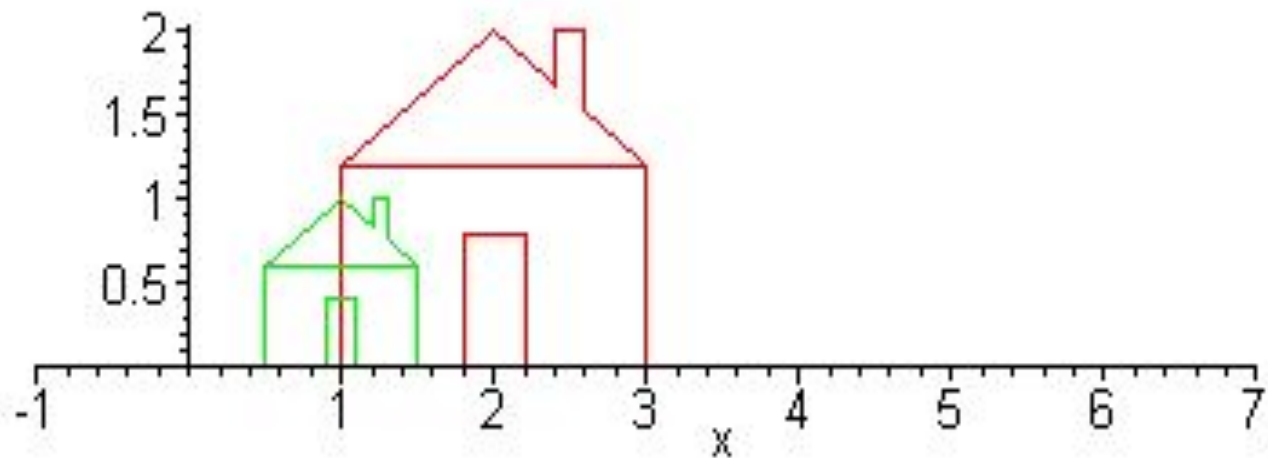
$$\begin{aligned} x' &= x + \alpha_x \\ y' &= y + \alpha_y \\ z' &= z + \alpha_z \end{aligned}$$

TRANSLATION

Inverse of Translation Matrix

$$T^{-1}(\alpha_x, \alpha_y, \alpha_z) = T(-\alpha_x, -\alpha_y, -\alpha_z) = \begin{pmatrix} 1 & 0 & 0 & -\alpha_x \\ 0 & 1 & 0 & -\alpha_y \\ 0 & 0 & 1 & -\alpha_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Scaling



SCALING

- Used to increase or decrease the size of the primitive.
- If a point P is moved to P' by scaling it by a factor S then $P' = SP$
- For both scaling and rotation, there is a fixed point that is unchanged by the transformation.
- The fixed point may be
 - Origin
 - Any pivot point

SCALING with origin as Fixed point

$$P' = S.P \quad \text{where} \quad S = \begin{pmatrix} \beta_x & 0 & 0 & 0 \\ 0 & \beta_y & 0 & 0 \\ 0 & 0 & \beta_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

S is called as Scaling Matrix.

SCALING

$$p' = \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \beta_x & 0 & 0 & 0 \\ 0 & \beta_y & 0 & 0 \\ 0 & 0 & \beta_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \beta_x \cdot x \\ \beta_y \cdot y \\ \beta_z \cdot z \\ 1 \end{pmatrix}$$

$$\begin{aligned} x' &= \beta_x \cdot x \\ y' &= \beta_y \cdot y \\ z' &= \beta_z \cdot z \end{aligned}$$

SCALING

Inverse of scaling matrix

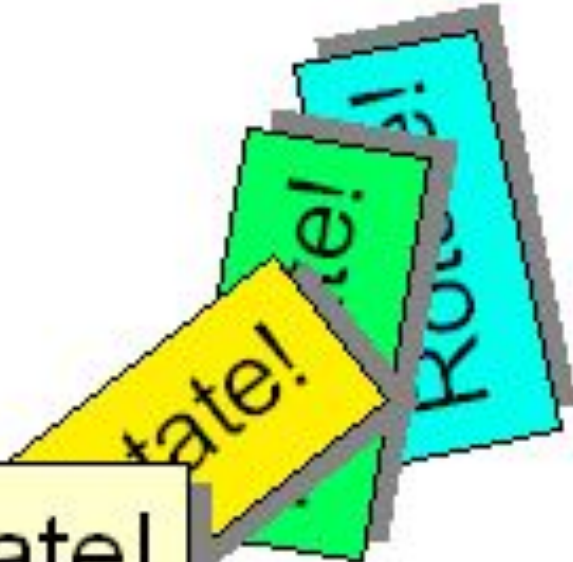
$$S^{-1}(\beta_x, \beta_y, \beta_z) = S(1/\beta_x, 1/\beta_y, 1/\beta_z) = \begin{pmatrix} 1/\beta_x & 0 & 0 & 0 \\ 0 & 1/\beta_y & 0 & 0 \\ 0 & 0 & 1/\beta_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation



SUNFLOWER

Rotate!



ROTATION

- Enables the programmer to rotate a given point wrt 3 degrees of freedom.
- **Note :**
 - Matrix multiplication is not commutative.
 - Rotation about x axis by angle θ followed by rotation about y axis by θ does not give the same result as the reverse order.

Rotation about z axis

$$p' = \begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} x \cos\theta - y \sin\theta \\ x \sin\theta + y \cos\theta \\ z \\ 1 \end{pmatrix}$$

Rotation about x and y axes

$$\mathbf{R} = \mathbf{R}_x(q) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D Rotation

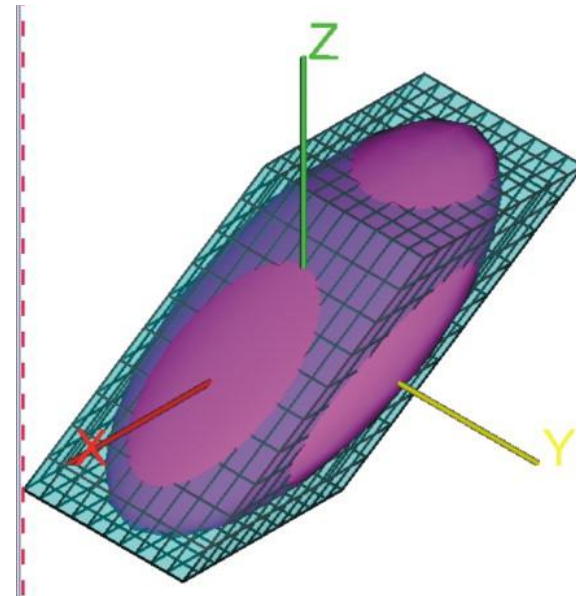
$$\mathbf{R} = \mathbf{R}_y(q) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about x and y axes

The inverse of rotation matrix can be obtained using

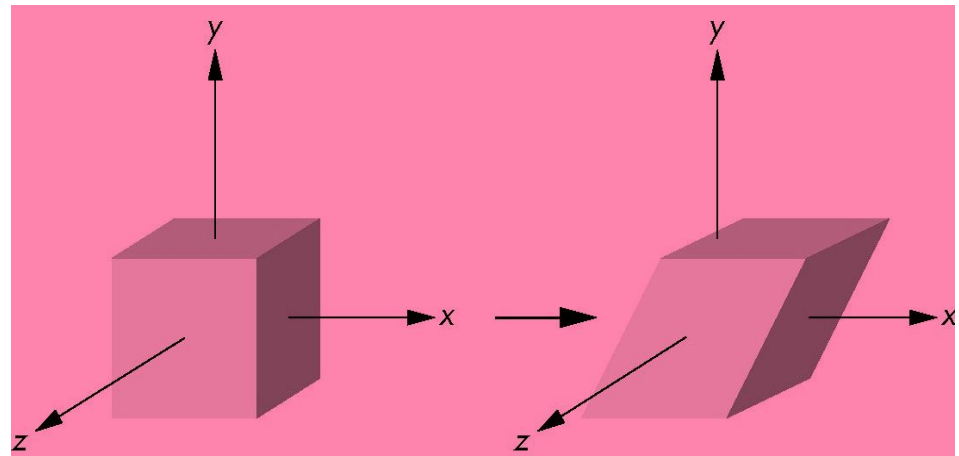
- $\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta)$
- $\cos(-\theta) = \cos\theta$
- $\sin(-\theta) = -\sin\theta$

Shear



Shear

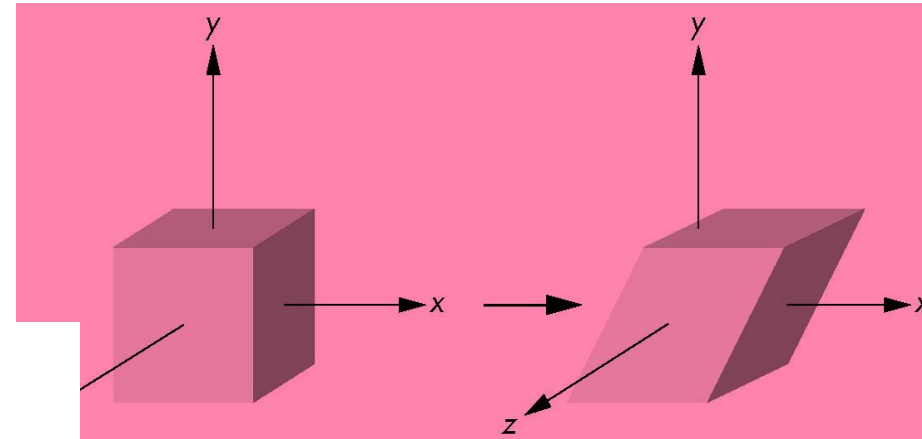
- Consider a cube centred at the origin, aligned with axes and viewed from +ve z-axis.
- Shear enables the programmer to pull the **top of the object** in one direction and **bottom of the object** in another direction.
- Object can be sheared in **x, y or z direction**.



Shear

Object is sheared in x direction (object // to x-axis)

- x is changed
- y is unchanged
- z is unchanged

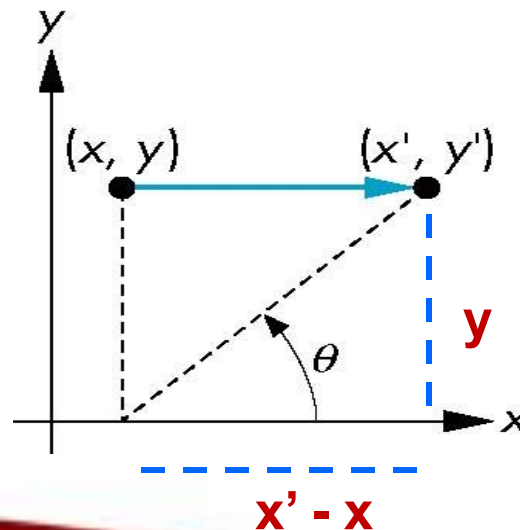


Equations

$$x' = x + y \cot\theta$$

$$y' = y$$

$$z' = z$$



$$\cot\theta = \text{adj/opp}$$

$$\cot\theta = (x' - x) / y$$

$$x' = x + y \cot\theta$$

Shear

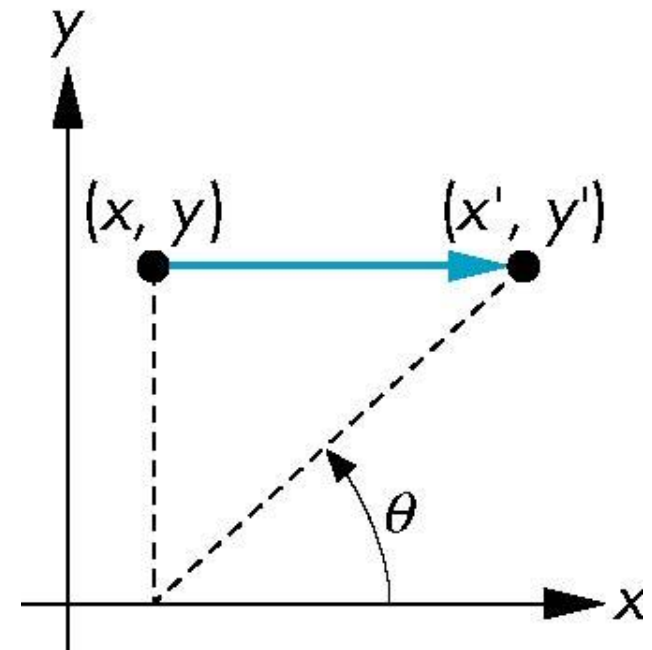
Object is sheared in x direction (object // to x-axis)

$$x' = x + y \cot \theta$$

$$y' = y$$

$$z' = z$$

$$H_x(q) = \begin{bmatrix} 1 & \cot \theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$\text{Inverse } H_x^{-1}(\theta) = H_x(\theta)$$

Shear

Example :

Object =

$\{ (2,1,1), (3,1,1), (3,2,1), (2,2,1) \}$

Shear along x-direction by 45°

Ans : $\cot 45 = 1$

$P_1' = (2+1*1), 1 = (3,1,1)$

$P_2' = (3+1*1), 1 = (4,1,1)$

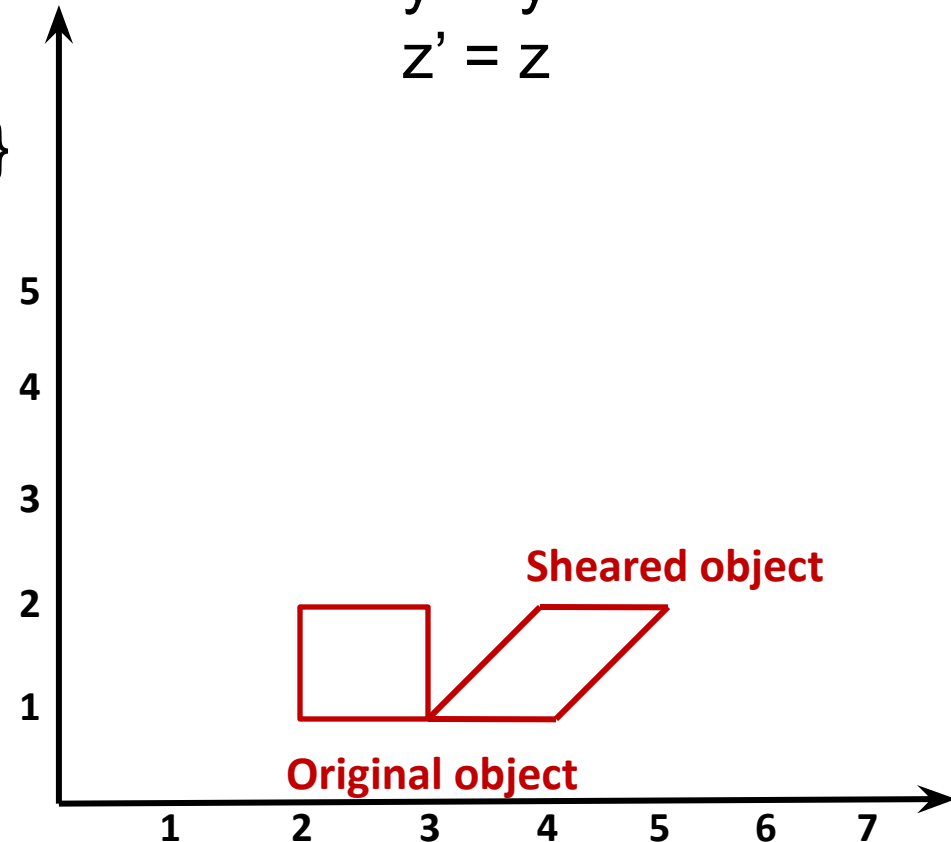
$P_3' = (3+2*1), 2 = (5,2,1)$

$P_4' = (2+2*1), 2 = (4,2,1)$

$$x' = x + y \cot \theta$$

$$y' = y$$

$$z' = z$$



Shear

Question1 :

Object =

$\{ (2,1,1), (3,1,1), (4,3,1), (2.5, 5,1), (1,3,1) \}$

Shear along x-direction by 60^0

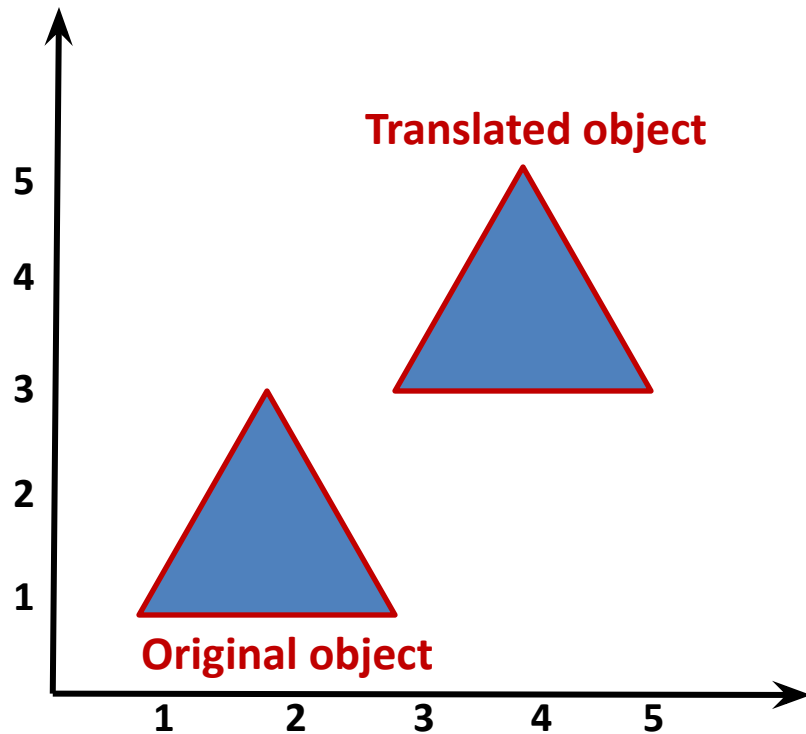
Question2 :

Object =

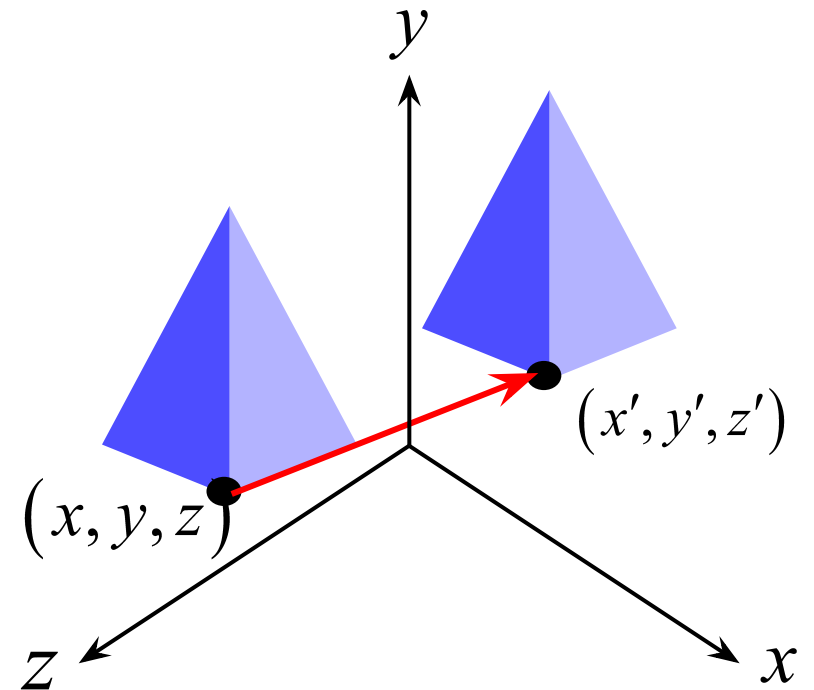
$\{ (1,1,1), (2,1,1), (1,2,1) \}$

Shear along x-direction by 30^0

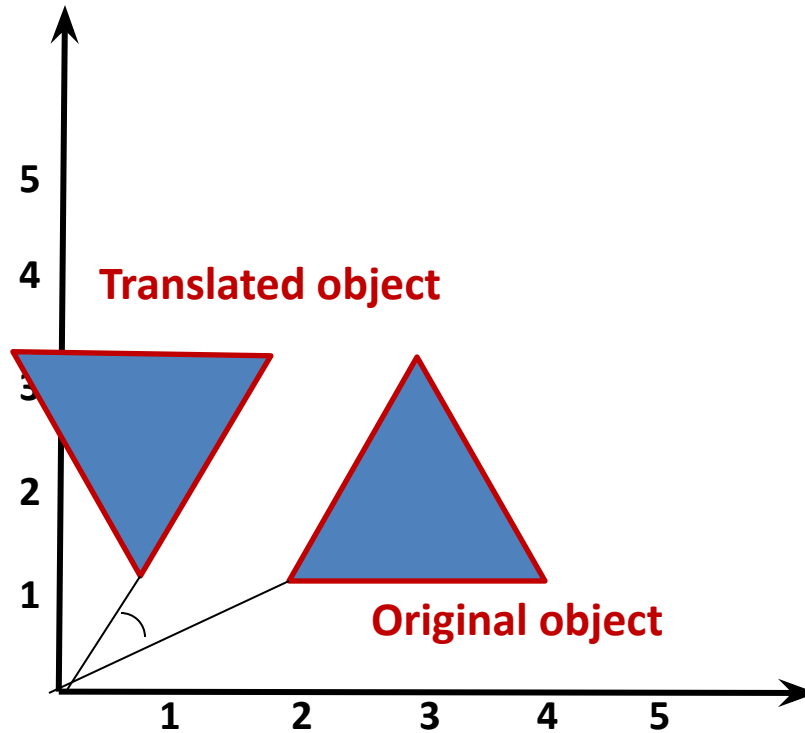
2D Translation



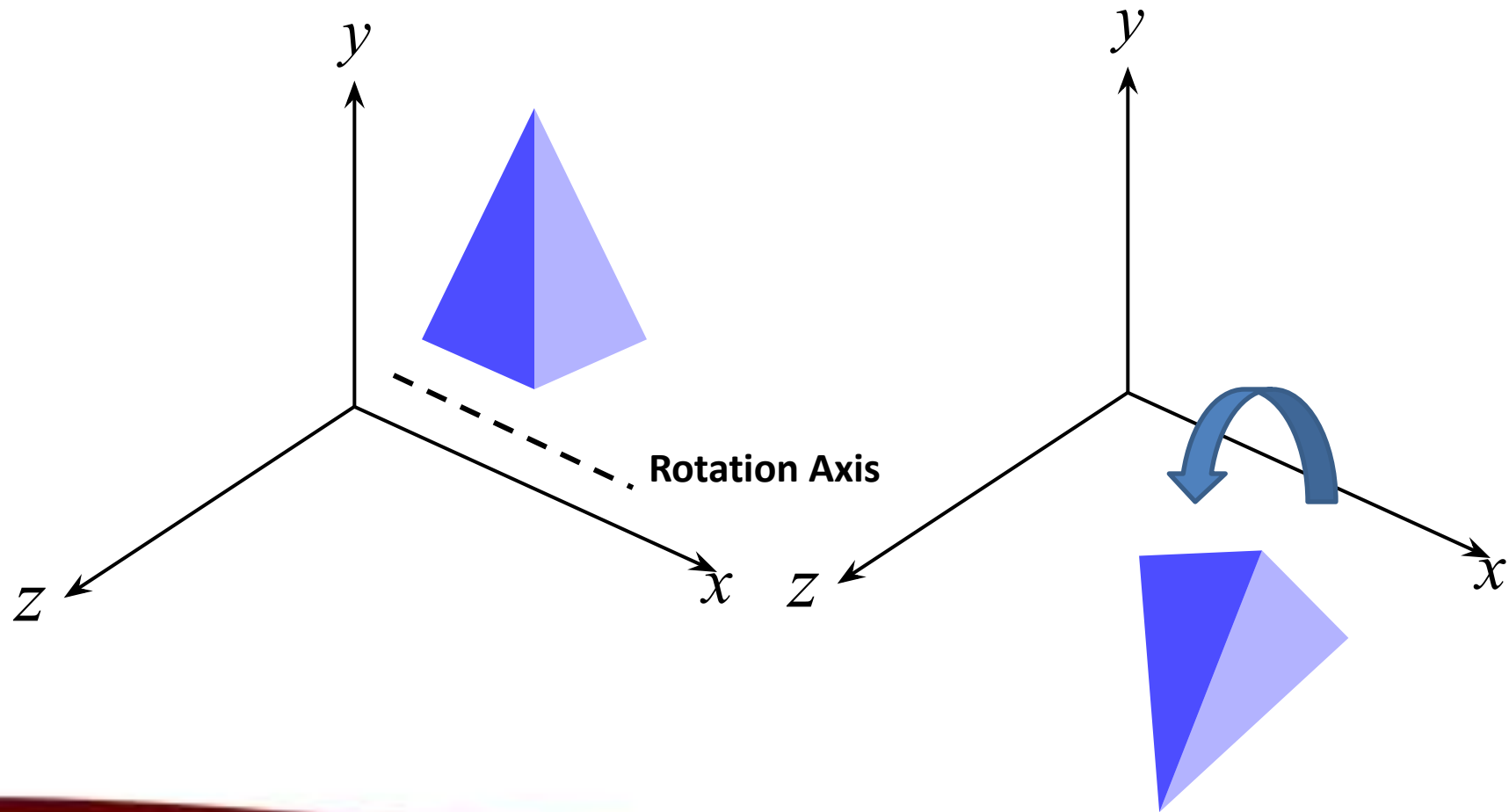
3D Translation



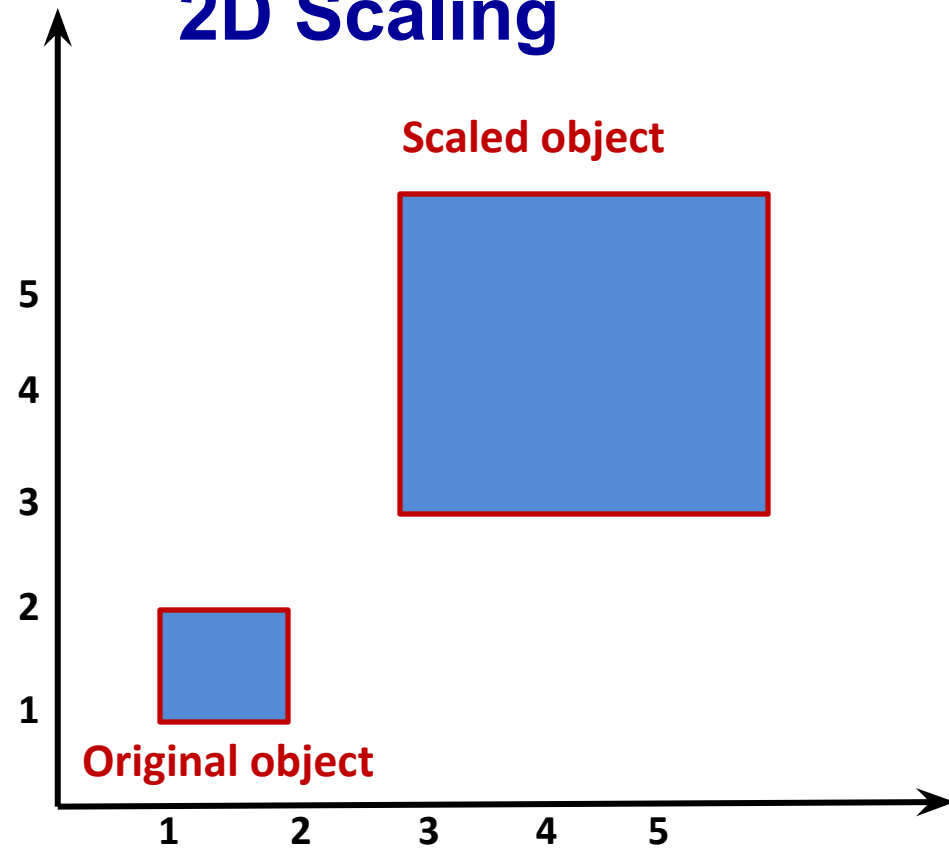
2D Rotation



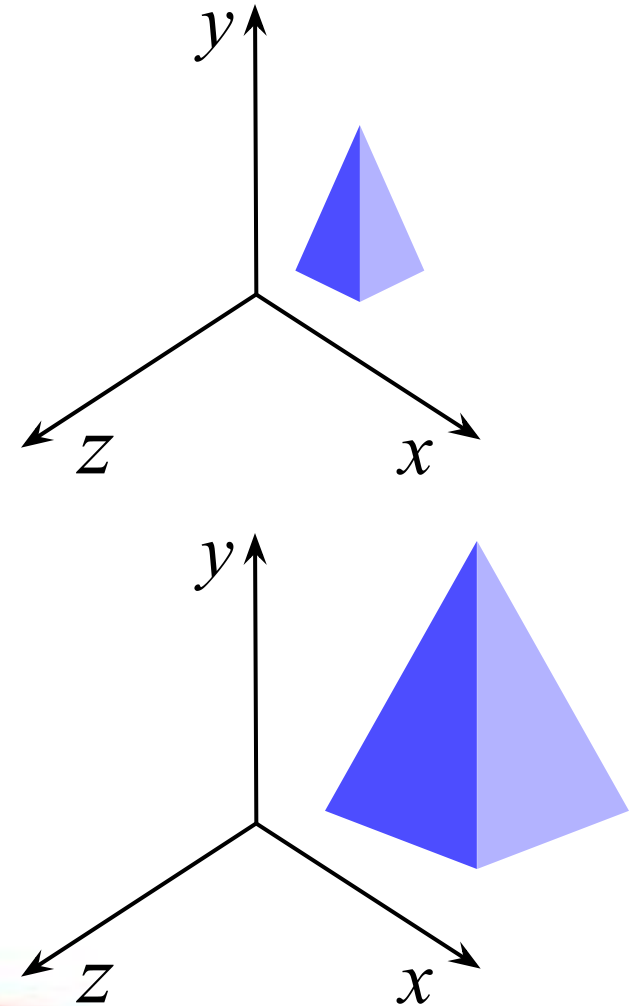
3D Rotation



2D Scaling



3D Scaling



3D Transformations

Transformation	Transformation matrix
Translation	$\begin{pmatrix} 1 & 0 & 0 & \alpha_x \\ 0 & 1 & 0 & \alpha_y \\ 0 & 0 & 1 & \alpha_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$
Scaling	$\begin{pmatrix} \beta_x & 0 & 0 & 0 \\ 0 & \beta_y & 0 & 0 \\ 0 & 0 & \beta_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

3D Transformations

Transformation	Transformation matrix
Rotation about <i>z axis</i>	$\begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
Rotation about <i>x axis</i>	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
Rotation about <i>y axis</i>	$\begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Concatenation of Transformation

$$\begin{bmatrix} x_{prev} \\ y_{prev} \\ 1 \end{bmatrix} = \begin{bmatrix} a_1 c_1 e_1 \\ b_1 d_1 f_1 \\ 0 \ 0 \ 1 \end{bmatrix} \cdot \begin{bmatrix} a_2 c_2 e_2 \\ b_2 d_2 f_2 \\ 0 \ 0 \ 1 \end{bmatrix} \cdot \begin{bmatrix} x_{curr} \\ y_{curr} \\ 1 \end{bmatrix}$$

Concatenation of Transformation

- It refers to *the multiplication of basic transformations* in order to produce arbitrary transformation.
- This approach fits well with pipeline architecture for implementing graphics system.

Concatenation of Transformation

- Concatenation

multiplication of basic transformation in order to produce arbitrary transformation.

- Successive transformations on point p and create new point q .

$$q = C B A p$$

A, B, C are arbitrary 4×4 Matrices.

- The order of transformation effects the efficiency of calculation.



Concatenation of Transformation

$p \rightarrow A \rightarrow B \rightarrow C \rightarrow q$

We carryout A, followed by B, followed by C.

$$q = (C (B (A p)))$$

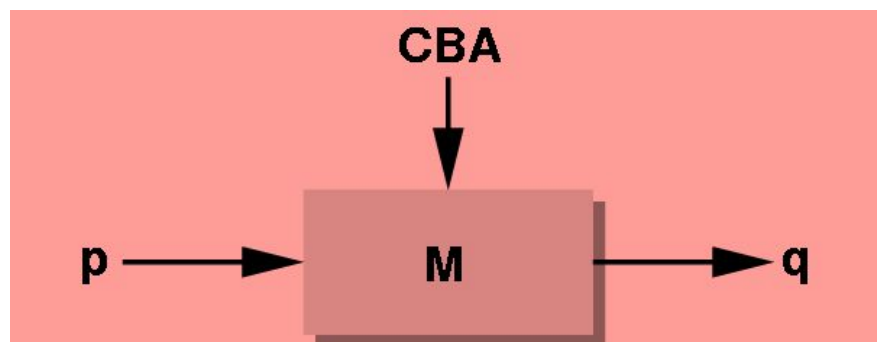


Concatenation of Transformation

If we have many points to transform, proceed in 2 steps.

1. Calculate $M = CBA$
2. $q = M.p$

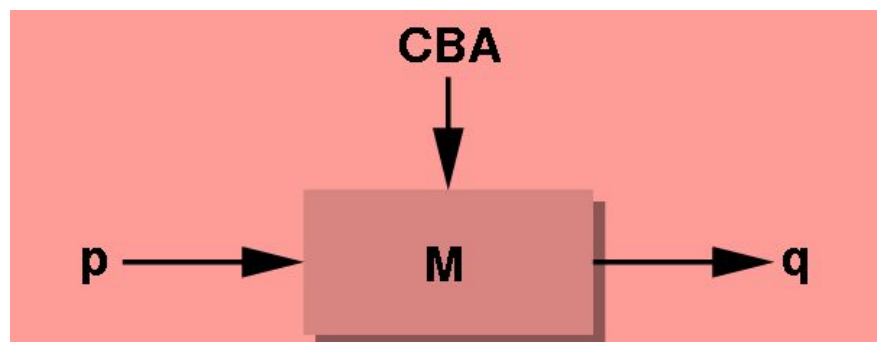
This order corresponds to pipeline architecture where we compute M first, then load it into a pipeline transformation unit.



Concatenation of Transformation

Advantages

- M is calculated only once and can be applied to thousands of points.
- So the burden of calculate single matrix multiplication is reduced.

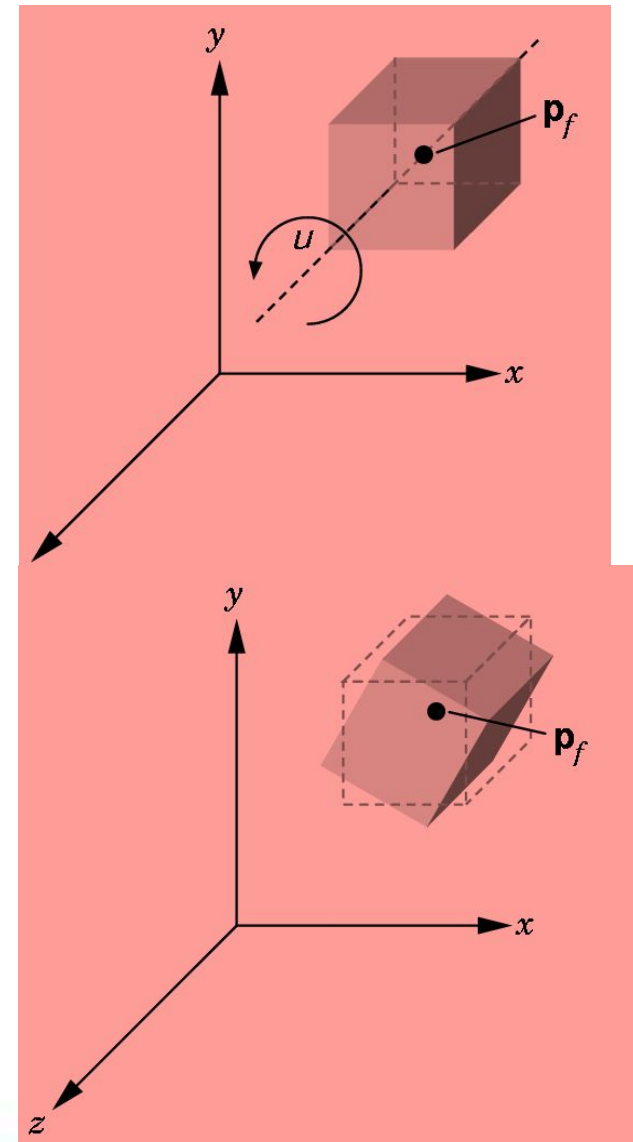


Calculation of M in Concatenation of Transformation

1. Rotation about a fixed point
2. General Rotation
3. Instance Transformation
4. Rotation about arbitrary(fixed) axis

1. Rotation about a fixed point

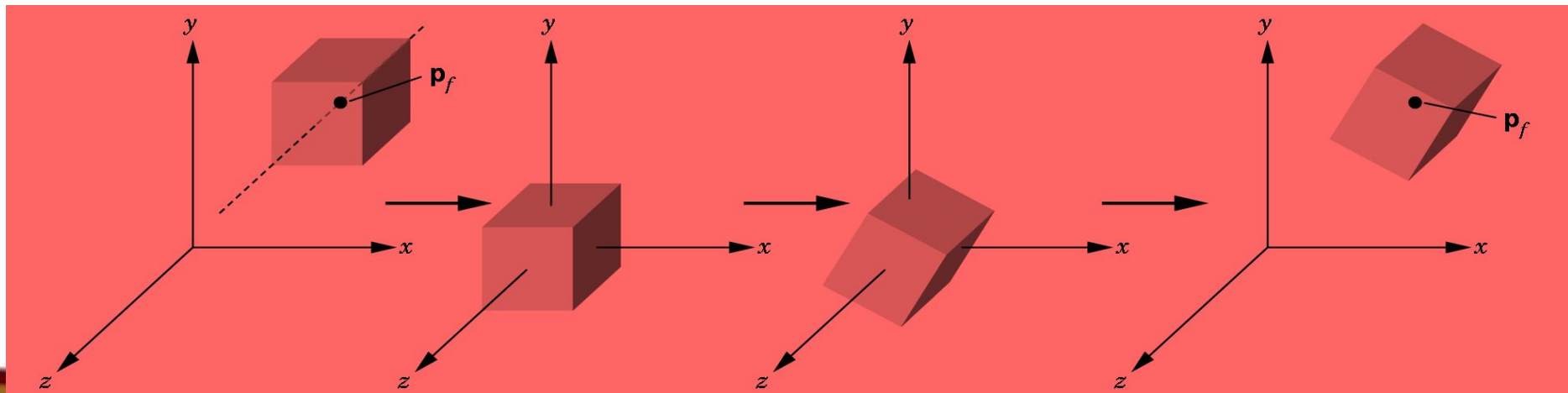
- Consider a cube with its centre at P_f and its sides aligned with the axes.
- We want to rotate the cube about z-axis, about its centre P_f .
- **P_f becomes fixed point of transformation.**



1. Rotation about a fixed point

If P_f were origin, we want to rotate through z axis, use $R_z(\theta)$.
Hence

- First move the cube to the origin ----> $T(-P_f)$
- Then apply $R_z(\theta)$ -----> $R_z(\theta)$
- Then move the object back such that its centre is again at P_f . -----> $T(P_f)$



1. Rotation about a fixed point

Concatenate them together, single matrix is obtained.

$$M = T(P_f) R_z(\theta) T(-P_f)$$

$$\begin{pmatrix} 1 & 0 & 0 & x_f \\ 0 & 1 & 0 & y_f \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -x_f \\ 0 & 1 & 0 & -y_f \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$T(P_f) \quad R_z(\theta) \quad T(-P_f)$

1. Rotation about a fixed point

Concatenate them together, single matrix is obtained.

$$M = T(P_f) R_z(\theta) T(-P_f)$$

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 & x_f \\ \sin\theta & \cos\theta & 0 & y_f \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -x_f \\ 0 & 1 & 0 & -y_f \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$T(P_f) \quad R_z(\theta) \qquad \qquad T(-P_f)$

1. Rotation about a fixed point

Concatenate them together, single matrix is obtained.

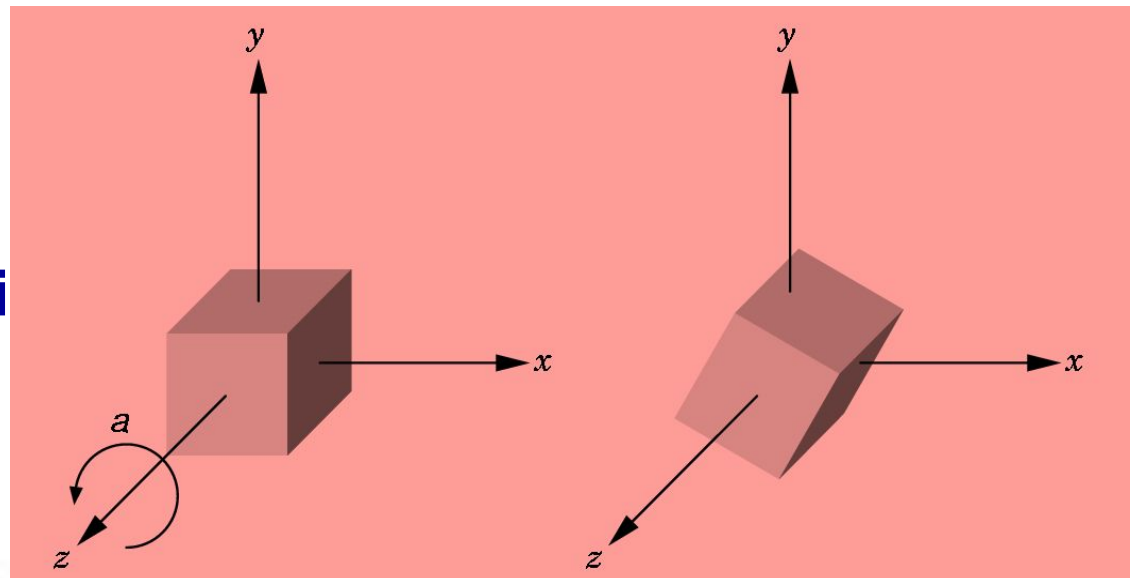
$$M = T(P_f) R_z(\theta) T(-P_f)$$

$$M = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & x_f - x_f\cos\theta + y_f\sin\theta \\ \sin\theta & \cos\theta & 0 & y_f - x_f\sin\theta - y_f\cos\theta \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2. General Rotation

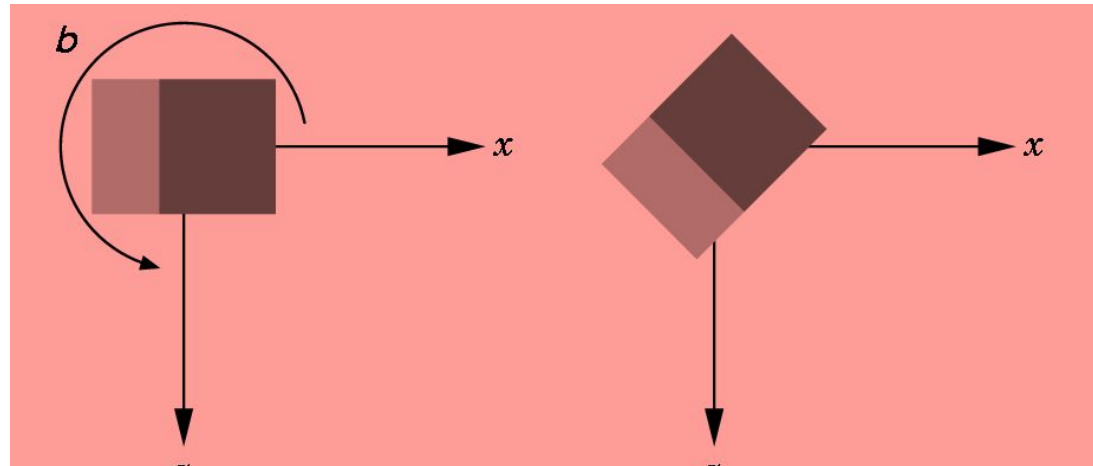
- Consider a cube centred at the origin with its sides aligned with the axes.
- Cube can be rotated about x , y , z axis.
- The transformation matrix is defined as $M = R_x R_y R_z$

Rotation about z-axis

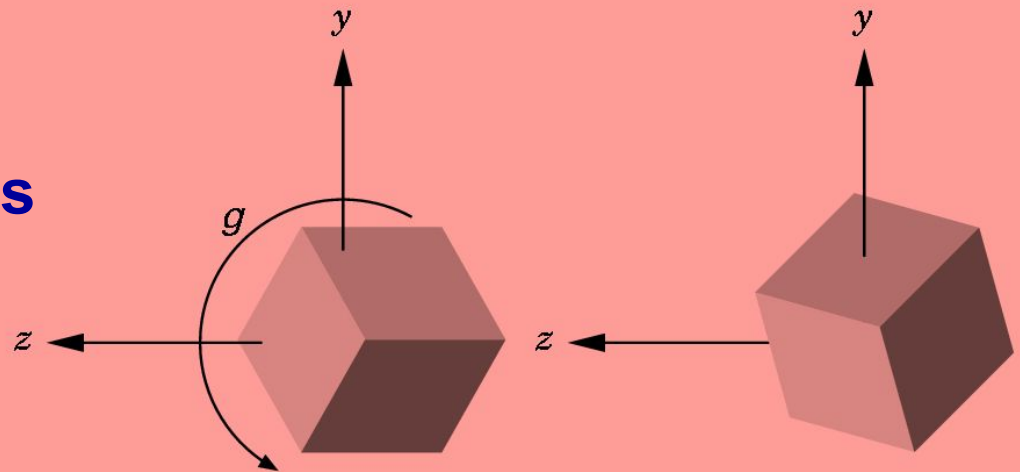


2. General Rotation

Rotation about y-axis



Rotation about x-axis



3. The instance Transformation

- Consider many simple objects as in Figure.
- There are two ways to create this scene.
 - Define each of these objects through the vertex using `glVertex3f()`
 - Define each of these object types **‘once’** at a convenient size, place and orientation.



3. The instance Transformation

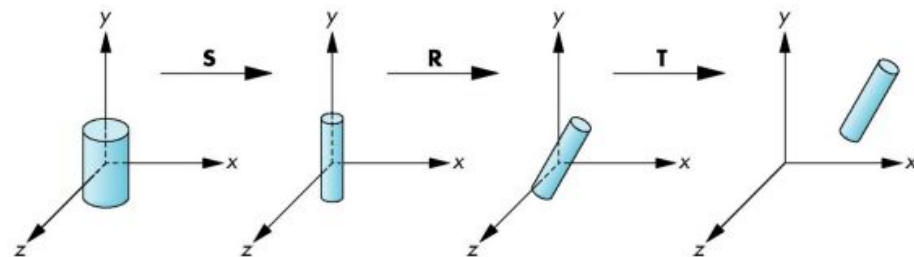
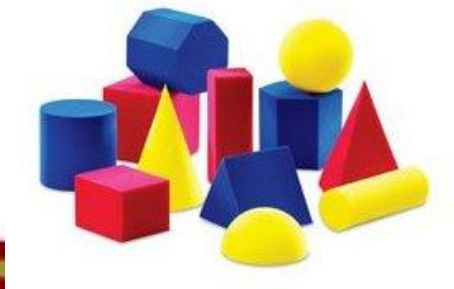
- Each occurrence of an object in the scene is an “instance” of that object’s prototype.
- The desired size, orientation and location can be obtained by applying affine transformations called instance transformations.



3. The instance Transformation

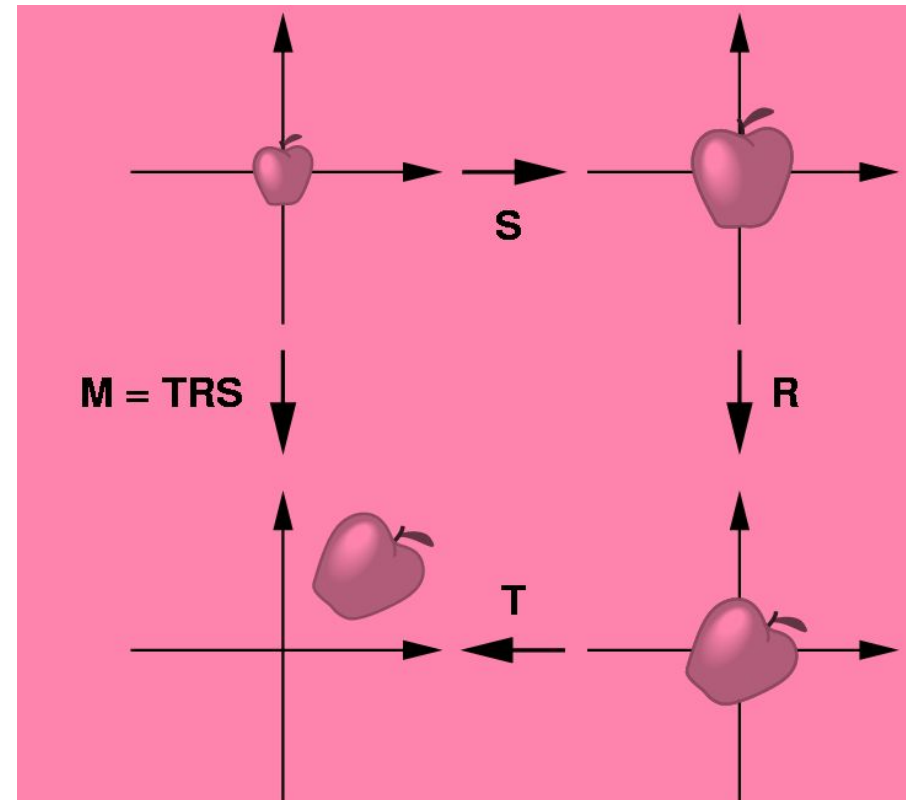
- Instance Transformation may be stored as a single call, in something like a subroutine.
- It contains a table consisting of each object and its transformation.

Symbol	Scale	Rotate	Translate
1	s_x, s_y, s_z	$\theta_x, \theta_y, \theta_z$	d_x, d_y, d_z
2			
3			
1			
1			
.			
.			



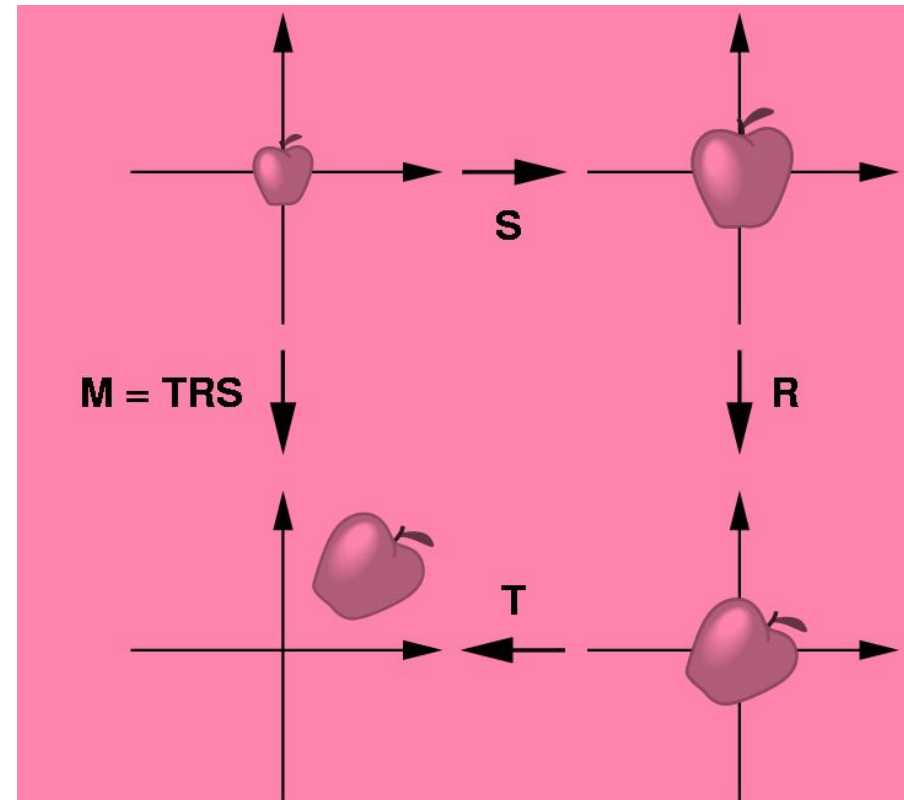
3. The instance Transformation

- Instance transformation is applied as shown.
- Objects are defined in their own frames with the origin at the centre and sides aligned with model frame axes.



3. The instance Transformation

- Apply an **instance transformation** to vertices to: Scale , Orient, Locate.
- Load object prototype in server.
- Send appropriate instance transform.

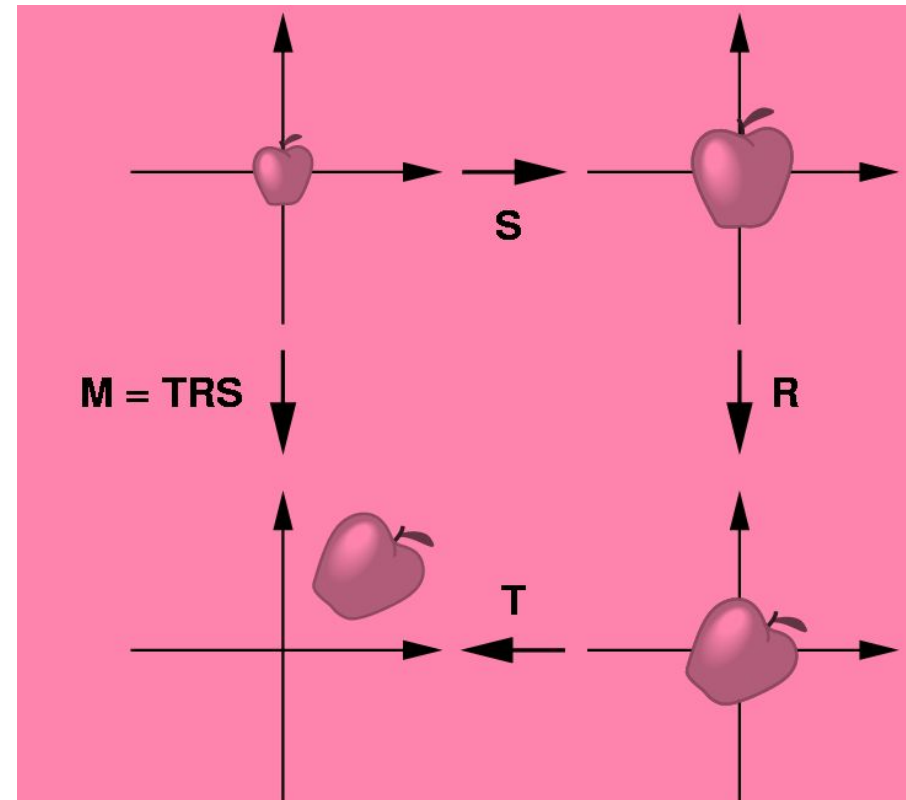


3. The instance Transformation

Steps of transformations

- Scale the object to the desired size.
- Rotate through rotation matrix.
- Translate it to the desired orientation.

Instance transformation is in the form **$M = TRS$**



4. Rotation about an arbitrary Axis

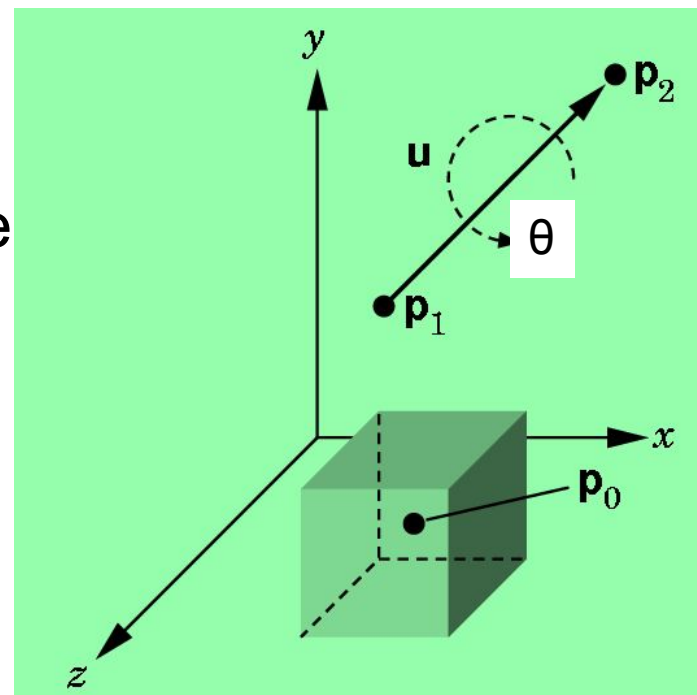
Consider a rotating cube.

Entities required for the rotation

- Fixed point P_0 (assume P_0 as the ce of the cube).
- A vector u about which we rotate cube.
- Angle of rotation θ

Note:

- The vector about which we wish to rotate the cube can be specified as $u = P_2 - P_1$



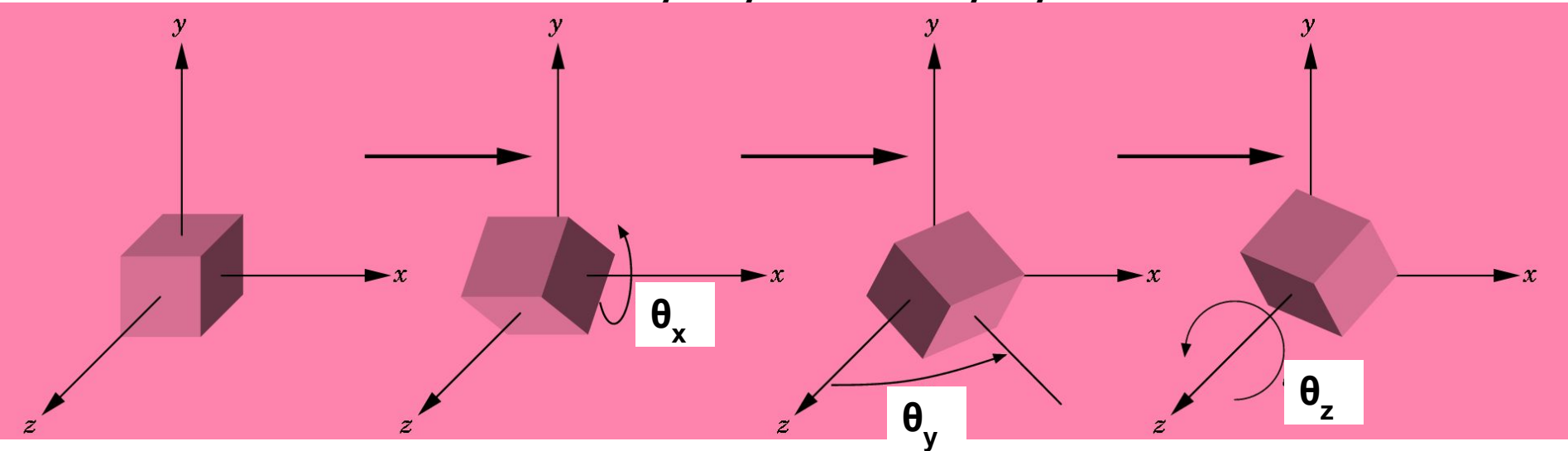
4. Rotation about an arbitrary Axis

Rotation of cube about vector u by θ

- Translate the fixed point P_0 **to the origin** by performing $T(-P_0)$
- Rotate the cube about x axis by performing $R_x(\theta_x)$
- Rotate the cube about y axis by performing $R_y(\theta_y)$
- Rotate the cube about z axis by angle θ $R_z(\theta)$
- Perform inverse rotation about y axis $R_y(-\theta_y)$
- Perform inverse rotation about x axis $R_x(-\theta_x)$
- Perform inverse translation at fixed point P_0 using $T(P_0)$

4. Rotation about an arbitrary Axis

$$R = R_x(-\theta_x) R_y(-\theta_y) R_z(\theta) R_y(\theta_y) R_x(\theta_x)$$



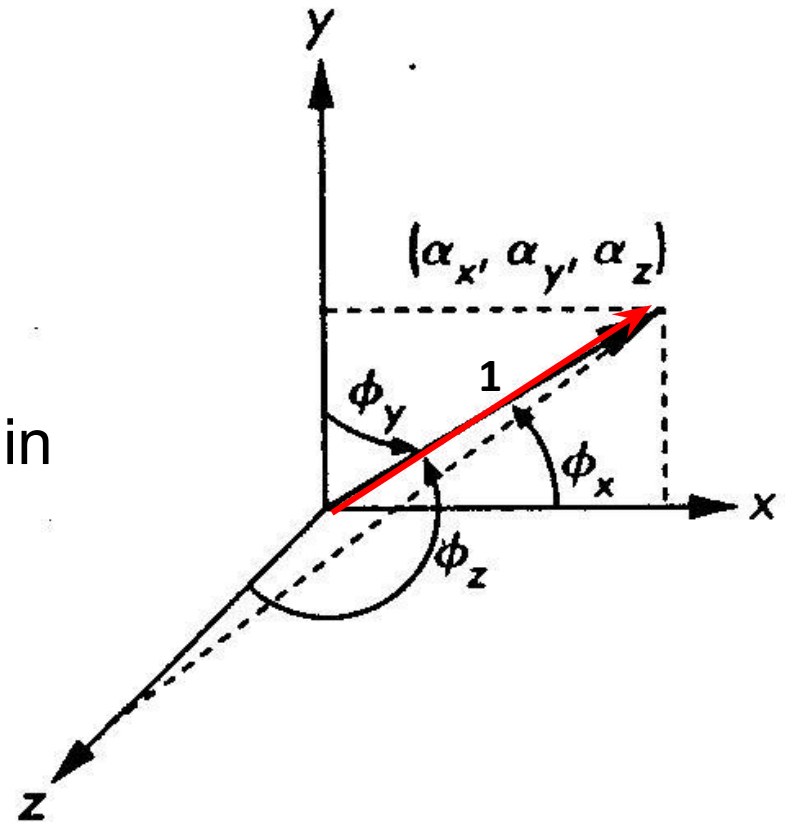
Concatenated Matrix M =

$$T(P_0) R_x(-\theta_x) R_y(-\theta_y) R_z(\theta) R_y(\theta_y) R_x(\theta_x) T(-P_0)$$

4. Rotation about an arbitrary Axis

Determining θ_x and θ_y

- Consider v as a unit length vector.
$$\alpha_x^2 + \alpha_y^2 + \alpha_z^2 = 1$$
- Draw a line segment from origin to the point $(\alpha_x, \alpha_y, \alpha_z)$.
- This line segment has
 - length=1 unit
 - orientation as v



4. Rotation about an arbitrary Axis

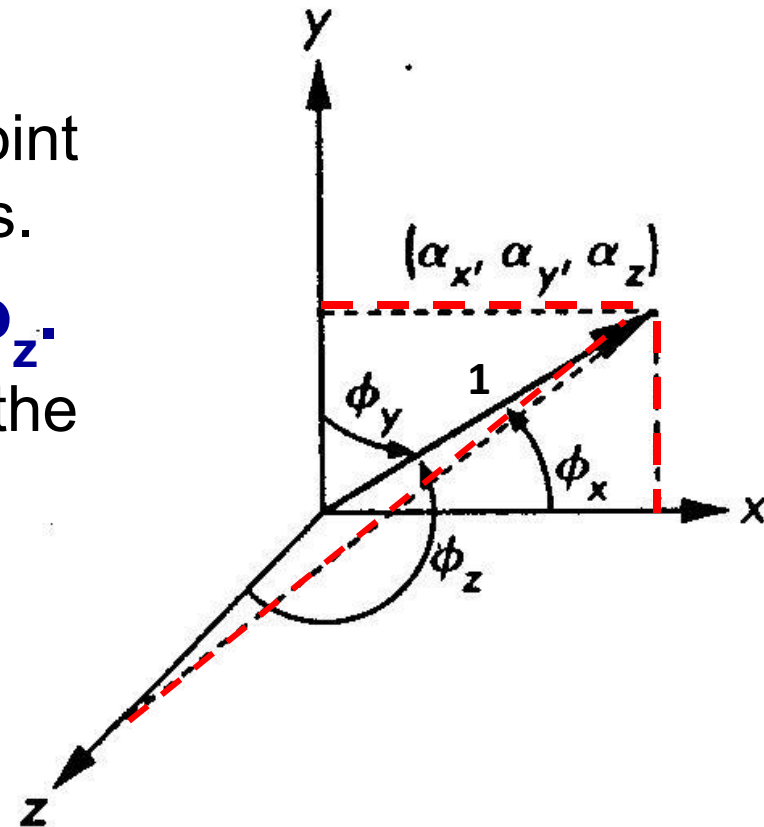
Determining θ_x and θ_y

- Draw a perpendicular from point $(\alpha_x, \alpha_y, \alpha_z)$ to the co-ordinate axes.
- The 3 direction angles ϕ_x, ϕ_y, ϕ_z are the angles between v and the axes.

$$\cos \phi_x = \alpha_x$$

$$\cos \phi_y = \alpha_y$$

$$\cos \phi_z = \alpha_z$$



4. Rotation about an arbitrary Axis

Computation of x rotation

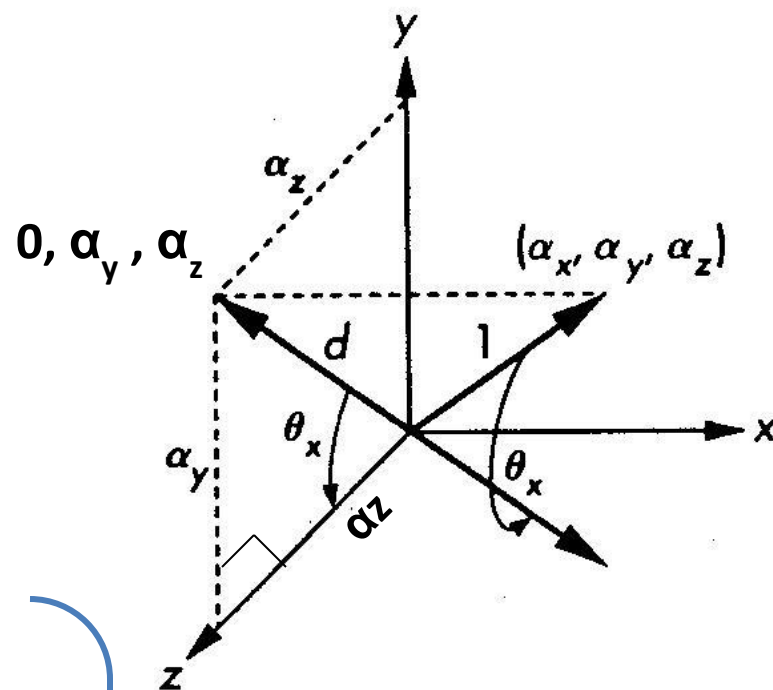
$$d = \sqrt{\alpha_y^2 + \alpha_z^2}$$

$$\cos\theta_x = \alpha_z/d$$

$$\sin\theta_x = \alpha_y/d$$

$$R_x(\theta_x) =$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \alpha_z/d & -\alpha_y/d & 0 \\ 0 & \alpha_y/d & \alpha_z/d & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



4. Rotation about an arbitrary Axis

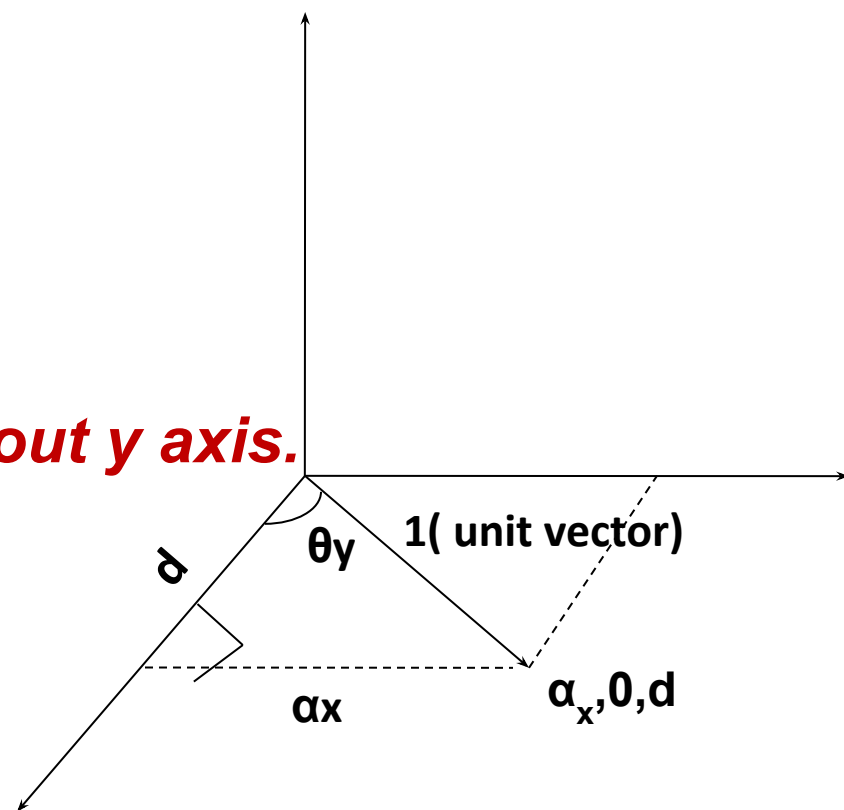
Computation of y rotation

$$d = \sqrt{\alpha_y^2 + \alpha_z^2}$$

$$\cos\theta_y = d/1, \quad \sin\theta_y = \alpha_x/1$$

Note : angle θ_y is clockwise about y axis.

$$R_y(\theta_y) = \begin{pmatrix} d & 0 & -\alpha_x & 0 \\ 0 & 1 & 0 & 0 \\ \alpha_x & 0 & d & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Concatenated Matrix M =

$$T(P_0) R_x(-\theta_x) R_y(-\theta_y) R_z(\theta) R_y(\theta_y) R_x(\theta_x) T(-P_0)$$

OpenGL Transformation Matrices

- We use model view matrix `glMatrixMode()` to select the matrix to which operations apply.
- In OpenGL, **model view matrix** is an affine transformation matrix.



Current Transformation Matrix **CTM**

Current Transformation Matrix (CTM)

- 4 x 4 homogeneous coordinate matrix.
- It is a matrix applied to any vertex that is defined subsequent to its setting.
- The CTM is **defined in the user program** and loaded into a transformation unit.
- If we change CTM, we change state of the system.
- CTM is the **part of the pipeline**.



Current Transformation Matrix (CTM)

- If p is a vertex specified in the application, pipeline produces Cp .
- CTM is a 4×4 matrix. it can be altered by a set of functions provided by graphics package.
- Denote CTM by C .
- Initially CTM is set to **identity matrix**.
- It can be reinitialized as needed.
- Use the symbol $\leftarrow$ to denote replacement. $C \leftarrow I$



Current Transformation Matrix (CTM)

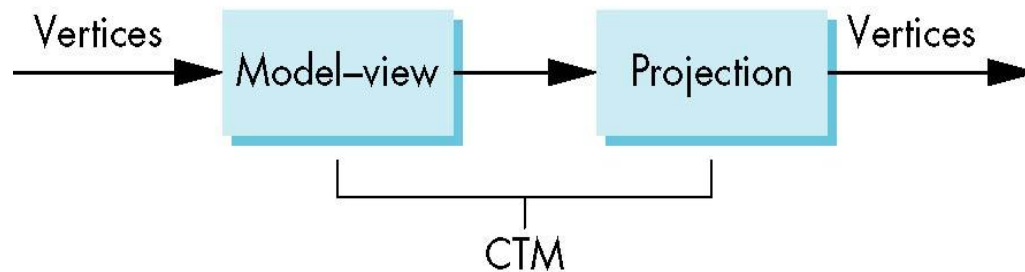
Functions that alter CTM

- Those that load it with some matrix.
- Those that modify it by premultiplication/postmultiplication by a matrix. **OpenGL uses postmultiplication.**

Transformation	Load form	Postmultiplication form
Translation	$C \leftarrow T$	$C \leftarrow CT$
Scaling	$C \leftarrow S$	$C \leftarrow CS$
Rotation	$C \leftarrow R$	$C \leftarrow CR$

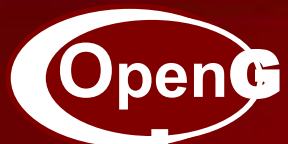
CTM in OpenGL

- In OpenGL, the matrix applied to all primitives is the **product of model view matrix and the projection matrix**.
- CTM is used as the product of these matrices.



CTM in OpenGL

- We can *manipulate individually* by selecting `glMatrixMode`.
- Matrix can be loaded with `glLoadMatrix(pointer_to_matrix)`.
- Matrix can be set to identity matrix by `glLoadIdentity()`.
- The selected matrix can be altered with `glMultMatrixf(pointer_to_matrix)`
- **Rotation** – `glRotatef(angle, vx, vy, vz)`
- **Translation** – `glTranslatef(dx,dy,dz)`
- **Scaling** – `glScalef(sx, sy, sz)`



Rotation about a fixed point in OpenGL - Example

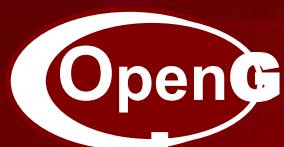
Degree of rotation : 45° about the line through the origin and the point (1,2,3) with a fixed point of (4,5,6)

1. Translate to origin $T(-Px)$
2. Rotate through angle θ $R_z(\theta)$
3. Translate it back to fixed point $T(Px)$

OpenGL functions

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(4.0,5.0,6.0);
glRotatef(45.0, 1.0,2.0,3.0);
glTranslatef(-4.0,-5.0,-6.0);
```

**Remember that
last matrix
specified in the
program is the first
applied.**



Order of Transformation

Rule in OpenGL

Transformation specified **last is the one applied first.**

Sequence of operations

$$C \leftarrow I$$

$$C \leftarrow CT(4.0, 5.0, 6.0)$$

$$C \leftarrow CR(45.0, 1.0, 2.0, 3.0)$$

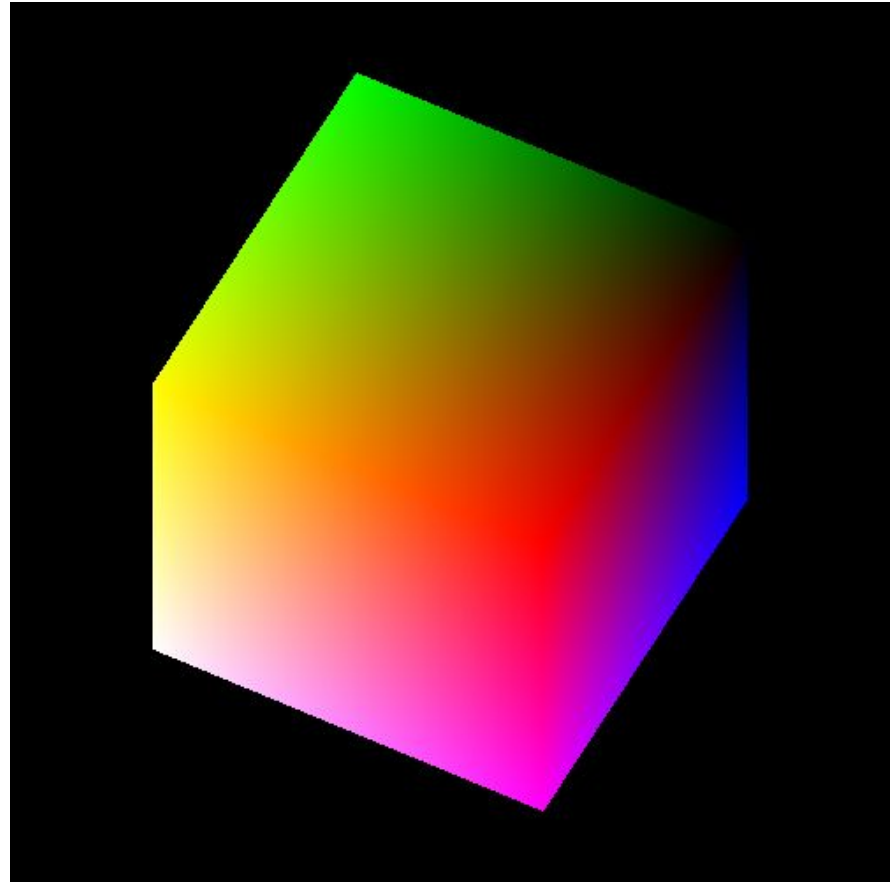
$$C \leftarrow CT(-4.0, -5.0, -6.0)$$

In each step, we postmultiply at the end of the existing CTM forming the matrix

$$C = T(4.0, 5.0, 6.0) R(45.0, 1.0, 2.0, 3.0) T(-4.0, -5.0, -6.0)$$

New vertex $q = Cp$

Spinning of the cube



Spinning of the cube

Explanation about

```
glutDisplayFunc(display);
```

```
glutIdleFunc(spincube);
```

```
glutMouseFunc(mouse);
```

```
void mykey(char key, int mousex, int mousey)
{
    if(key=='q' || key=='Q') exit(0);
}
```


Spinning of the cube

```
void display()
```

```
{
```

```
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

```
    glLoadIdentity();
```

```
    glRotatef(theta[0], 1.0, 0.0, 0.0);
```

```
    glRotatef(theta[1], 0.0, 1.0, 0.0);
```

```
    glRotatef(theta[2], 0.0, 0.0, 1.0);
```

```
    colorcube();
```

```
    glFlush();
```

```
    glutSwapBuffers();
```

```
}
```

Spinning of the cube

```
void spinCube()
```

```
{  
    theta[axis] += 0.05; // Controls the speed  
    if( theta[axis] > 360.0 ) theta[axis] -= 360.0;  
    glutPostRedisplay();  
}
```

```
void mouse(int btn, int state, int x, int y)
```

```
{  
    if(btn==GLUT_LEFT_BUTTON && state == GLUT_DOWN) axis = 0; // x  
    if(btn==GLUT_MIDDLE_BUTTON && state == GLUT_DOWN) axis = 1;  
    if(btn==GLUT_RIGHT_BUTTON && state == GLUT_DOWN) axis = 2;  
}
```

Loading, pushing and popping matrices

- Perform a transformation and then return to the same state as before its execution.
- This situation occurs when we execute an instance transformation that applies to a particular object and does not apply to the following objects in the code.
- Rather than recomputing the transformation, push the transformation matrix on a stack with `glPushMatrix()`.
- Recover it by `glPopMatrix()`.



Interfaces to 3D Applications



Interfaces to 3D Applications

GLUT provides smoother interface to 3D applications by allowing the user to use keyboard along with mouse.

Areas of Screen

- Suppose the user wants to use
 - One mouse button for orientation of the object
 - One for zoom in or zoom out of the object
 - One for translation of object to left or right

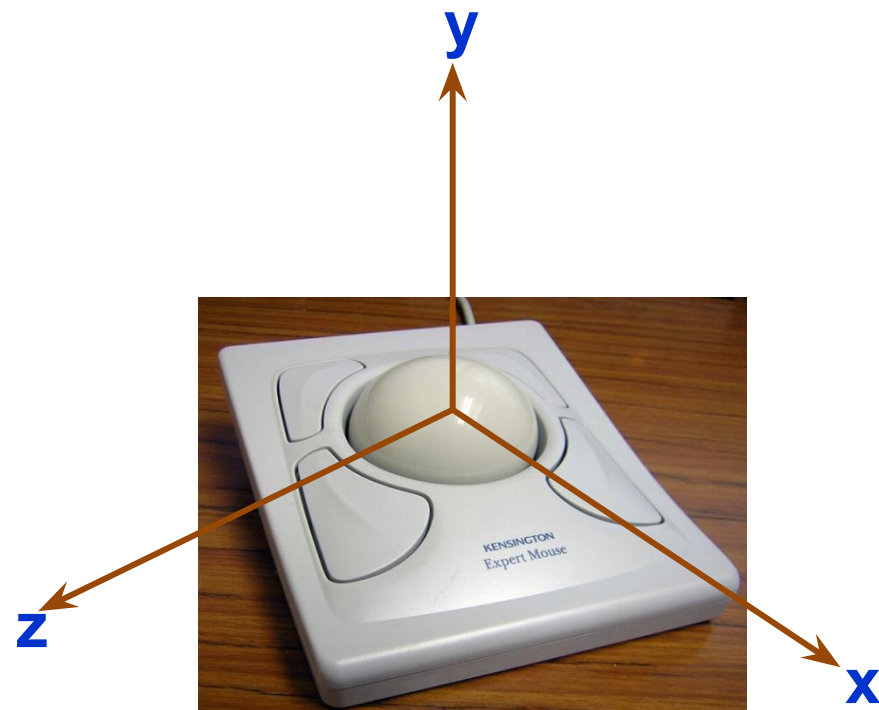
We can use motion callback to achieve these functions.

- The callback **returns the button which is been activated** and the **location of the mouse**.
- This location of mouse can be used to control the direction of rotation, translation and zoom in and out.

Interfaces to 3D Applications

Virtual track ball

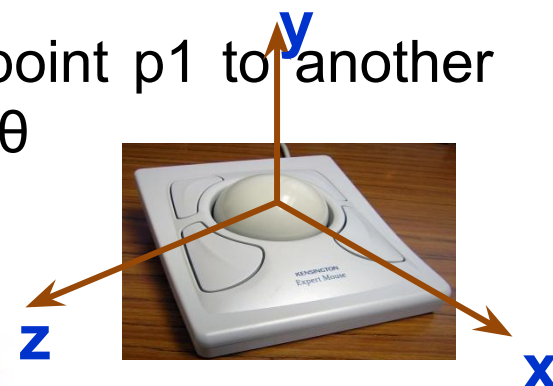
- Can be created using a mouse and the display device.
- **Advantage of virtual device**
 - Creates a **frictionless trackball** which once started to rotate will continue until stopped by the user.
 - Supports continuous rotations of objects but still allows changes in the speed and orientation.



Interfaces to 3D Applications

Virtual track ball

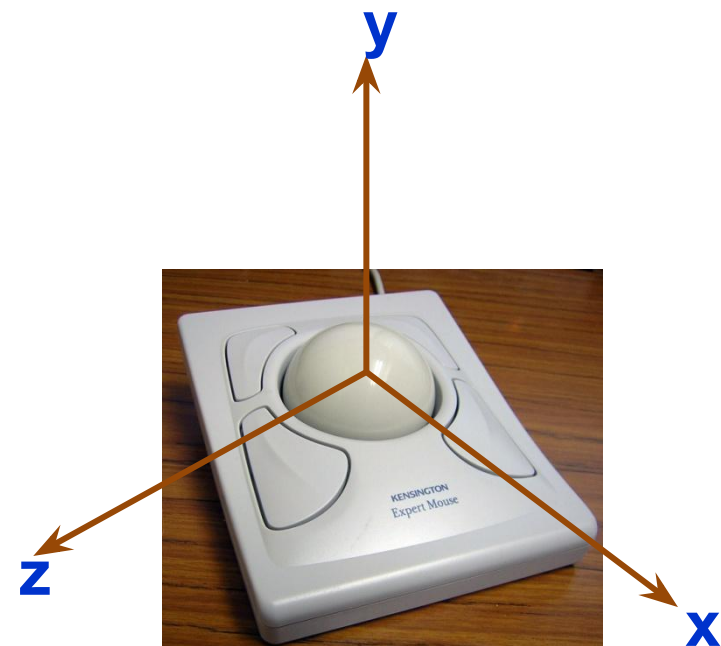
- Can be achieved by mapping the position of the trackball to that of a mouse.
- Assume that the trackball has a radius of 1 unit. We can map the position on its surface to the plane $y=0$ by doing orthogonal projection.
- The position (x,y,z) on the surface the ball is mapped to $(x,0,z)$ on the plane.
- The motion of the ball that moves from one point p_1 to another point p_2 can be computed $n=p_1 * p_2$ and $\sin\theta \approx \theta$



Interfaces to 3D Applications

Virtual track ball

- We can implement the virtual trackball through the use of idle, motion and mouse callbacks in GLUT.
- The process can be with 3 logical variables
 - **bool** trackingmouse=false;
 - **bool** trackballmove = false;
 - **bool** redrawContinue = false;
- Changes in these variables are controlled through the mouse callback.



Interfaces to 3D Applications

Incremental Rotation

- Glut provides for smooth incremental rotations.
- Given 2 orientations of the camera, we want to move smoothly from one orientation to other, the code is

```
for(i=0;i<imax;i++)  
{  
    glRotatef(delta_theta, dx, dy, dz);  
    drawobject();  
}
```

Interfaces to 3D Applications

Incremental Rotation

- **Problem of the code** : Calculation of rotation matrix requires evaluation of sines and cosines of 3 angles.
- To overcome this, compute rotation matrix once and reuse it through the following code.

```
GLfloat m[16];  
glRotatef(dx, dy, dz, delta_theta);  
glGetFloatv(GL_MODELVIEW_MATRIX, m);  
for(i=0;i<max;i++)  
{  
    glMultMatrixf(m);  
    draw_object();  
}
```

- We could also use small angle approximations
 $\sin\theta \approx \theta$ if θ is very small
 $\cos\theta \approx 1$

Interfaces to 3D Applications

Arbitrary rotation matrix with angle ψ along z axis, ϕ along y axis and θ along x axis can be achieved using

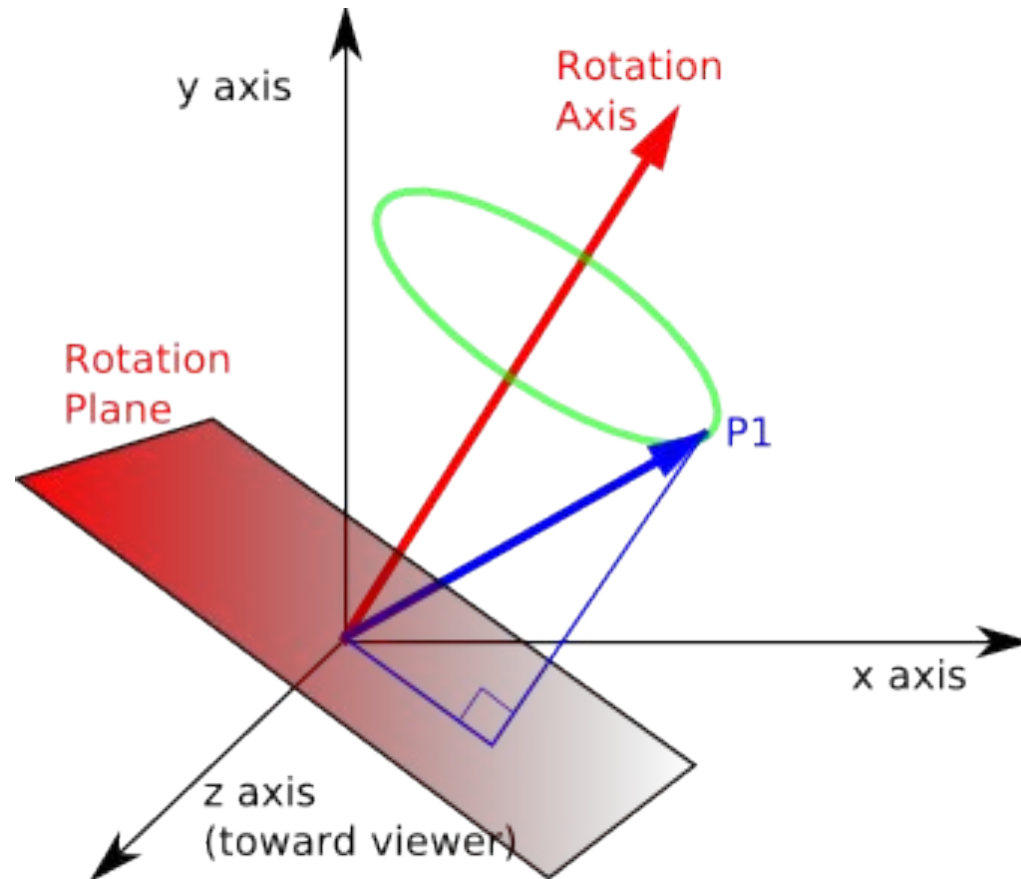
$$\mathbf{R} = \mathbf{R}_z(\psi) \mathbf{R}_y(\phi) \mathbf{R}_x(\theta)$$

$$\begin{pmatrix} \cos\psi & -\sin\psi & 0 & 0 \\ \sin\psi & \cos\psi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos\phi & \sin\phi & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\phi & \cos\phi & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

For small angles of θ, ϕ, ψ we get $\mathbf{R} \approx$

$$\begin{pmatrix} 1 & -\psi & \phi & 0 \\ \psi & 1 & -\theta & 0 \\ \theta & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Quaternions



QUATERNIONS

- Extension of complex numbers.
- Provide an alternative method for **describing and manipulating rotations**.
- Provide advantages for animation and hardware implementation of rotation.
- Two methods of performing rotations
 - Post multiplication of CTM with rotation matrix
 - Performing rotations using quaternions
- **Advantage of quaternion** : requires very little computations for rotations compared to rotation matrices.

QUATERNIONS

Euler's identity

$$e^{i\theta} = \cos\theta + i \sin\theta$$

Polar representation of complex number

$$c = a + ib = re^{i\theta} \text{ where } r = \sqrt{a^2 + b^2}$$
$$\theta = \tan^{-1} (b/a)$$

QUATERNIONS

Complex numbers versus quaternions

Definition (Complex numbers)

- ▶ Define basis elements 1 and i ;
- ▶ Define **complex numbers** as a vector space over reals: elements have the form $x + iy$;
- ▶ One more axiom required: $i^2 = -1$.

Definition (Quaternions)

- ▶ Define basis elements 1, i , j , k ;
- ▶ Define **quaternions** as a vector space over reals: elements have the form $q_0 + q_1i + q_2j + q_3k$;
- ▶ One more axiom:

$$i^2 = j^2 = k^2 = ijk = -1$$

QUATERNIONS

If denote c about the origin by φ to c' , the we can find c'

- using a rotation matrix
- we can use polar representation to write

$$c' = r e^{i(\theta+\varphi)} = r e^{i\theta} e^{i\varphi}$$

$e^{i\varphi}$ = rotation operator in the complex plane.

QUATERNIONS

- Specifying a rotation in 3D space involves two things
 - direction of rotation(vector)
 - amount of rotation(scalar)
- Since a quaternion has both scalar and vector representation, it can be used to specify rotation efficiently.
- If denote c about the origin by φ to c' , then we can find c' using a rotation matrix
- We can use polar representation to write $c' = r e^{i(\theta+\varphi)} = r e^{i\theta} e^{i\varphi}$
- $e^{i\varphi}$ = rotation operator in the complex plane.

QUATERNIONS

This is represented as quaternion.

$$a = (q_0, q_1, q_2, q_3) = (q_0, \mathbf{q}) \text{ where } \mathbf{q} = (q_1, q_2, q_3)$$

Properties of complex numbers

$$i^2 = j^2 = k^2 = ijk = -1$$

$$ij=k \quad ji=-k$$

$$jk=i \quad kj=-i$$

$$ki=j \quad ik=-j$$

$$a=(q_0, \mathbf{q})$$

$$b=(p_0, \mathbf{p})$$

$$a+b = (p_0+q_0, \mathbf{p}+\mathbf{q})$$

$$ab = (p_0q_0 - \mathbf{q}\cdot\mathbf{p}, q_0\mathbf{p} + p_0\mathbf{q} + \mathbf{q}\times\mathbf{p})$$

$$|a|^2 = q_0^2 + q_1^2 + q_2^2 + q_3^2 = q_0^2 + \mathbf{q}\cdot\mathbf{q}$$

$$a^{-1} = (1/|a|^2) (q_0, -\mathbf{q})$$

Quaternions & Rotation

- A mathematical object is defined.
- Relate it to geometric entities and see how it is used to carry out operations such as rotations.
- Vector part of quaternion represents a point in space $p = (0, p)$
- Components of $p = (x, y, z)$ give the location of point.
- Quaternion : **$r = (\cos \theta/2, \sin \theta/2 \ v)$** where v has unit length
- We can show that quaternion r is a unit quaternion ($|r|=1$)
- $r^{-1} = (\cos \theta/2, -\sin \theta/2 \ v)$

Quaternions & Rotation

- A point p in space can be represented in the quaternion representation as $p = (0, p)$
- Consider quaternion in the polar form :
 $r = (\cos \theta/2, \sin \theta/2 \ v)$ where v has unit length.
- $r^{-1} = (\cos \theta/2, -\sin \theta/2 \ v)$
- The new point p' after rotation can be obtained using quaternion product $p' = r p r^{-1}$
- **$p' = (x \cos \theta - y \sin \theta, x \sin \theta + y \cos \theta, z)$**

Quaternions & Rotation

if $v = (v_x, v_y, v_z)$

$$r = e^{\theta/2} (v_x i + v_y j + v_z k)$$

$$r = \cos \theta/2 + (v_x i + v_y j + v_z k) \sin \theta/2$$

$$r^{-1} = e^{-\theta/2} (v_x i + v_y j + v_z k)$$

$$r^{-1} = \cos \theta/2 - (v_x i + v_y j + v_z k) \sin \theta/2$$

rotation about arbitrary point $p' = r p r^{-1}$

$p' = (x \cos \theta - y \sin \theta, x \sin \theta + y \cos \theta, z)$ for normal rotation

Quaternions

Advantages

- They are simple to compose.
- They are numerically stable.
- They are more efficient.
- Representation is more compact than in matrix form.
- For a given axis and angle, it is easier to construct quaternion.
- For a given quaternion, one can easily read the axis and angle.
- ***Smooth rotations are possible.***

Solved Examples

1. Prove that two scaling transformations are commutative

ie, $S_1 S_2 = S_2 S_1$

$$\text{Let } S_1 = \begin{pmatrix} S_{x1} & 0 \\ 0 & S_{y1} \end{pmatrix}$$

$$\text{Let } S_2 = \begin{pmatrix} S_{x2} & 0 \\ 0 & S_{y2} \end{pmatrix}$$

$$S_1 S_2 = \begin{pmatrix} S_{x1} S_{x2} & 0 \\ 0 & S_{y1} S_{y2} \end{pmatrix}$$

$$S_2 S_1 = \begin{pmatrix} S_{x2} S_{x1} & 0 \\ 0 & S_{y2} S_{y1} \end{pmatrix}$$

Solved Examples

2. Prove that two 2D rotation about origin is

i) Commutative ie, $R_1 R_2 = R_2 R_1$

ii) Additive ie, $R_{\theta_1} R_{\theta_2} = R_{(\theta_1 + \theta_2)}$

i) Let $R_1 = \begin{pmatrix} \cos\theta_1 & -\sin\theta_1 \\ \sin\theta_1 & \cos\theta_1 \end{pmatrix}$

Let $R_2 = \begin{pmatrix} \cos\theta_2 & -\sin\theta_2 \\ \sin\theta_2 & \cos\theta_2 \end{pmatrix}$

$$R_1 \cdot R_2 = R_2 \cdot R_1$$

Solved Examples

2. Prove that two 2D rotation about origin is

ii) Additive ie, $R_{\theta_1} R_{\theta_2} = R(\theta_1 + \theta_2)$

$$\text{ii) Let } R(\theta_1 + \theta_2) = \begin{pmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) \end{pmatrix}$$

Let $R_{\theta_1} R_{\theta_2} = ?$

$$R_{\theta_1} R_{\theta_2} = R(\theta_1 + \theta_2)$$

Question Bank

1. Write an OpenGL program that allows to orient the cube with one mouse button, to translate it with a second and to 200m in and out with a third.

July 2009/10 marks

2. What are quaternions? With illustrative example, explain how quaternions are used in the rotations in a 3D space. Give the mathematical representations of quaternions.

July 2009/10 marks

3. What is a homogeneous co-ordinate system? Using this co-ordinate system, represent all the basic 2D transformations.

Jan 2010/12 marks

4. Write an OpenGL program to rotate a cube about x, y, and z axes. Use mouse buttons to select axis of rotation. Use `glRotatef()` function.

Jan 2010/8 marks

Question Bank

5. Explain the basic transformations in 3D and represent them in matrix form.

June 2010/7 marks

6. What is concatenation? How does it affect the efficiency of transformations?

June 2010/4 marks

7. What are the entities required to perform a rotation? Show the two rotations about the same axis commute.

June 2010/7 marks

8. What are the advantages of quaternions?

June 2010/2 marks

9. Explain how quaternions are used in rotation in a three dimension space.

Dec 2010/10 marks

Question Bank

10. Write a program rotating cube with viewer movement.

Dec 2010/10 marks

11. What is concatenation of transformations? Derive concatenated final matrix M for rotating a 3D object about a fixed point.

June 2011/8 marks

12. Consider a 3D cube object, with fixed point is at the centre of the cube and the angle of rotation θ about an arbitrary axis defined by two points P_1 and P_2 defining the vector u . Find the final rotation matrix R .

June 2011/12 marks

13. How do you achieve affine transformations by concatenation? Discuss.

Dec 2011/10 marks

14. What are quaternions? With an example, explain its mathematical representations. **Dec 2011/10 marks**

Question Bank

15. Define and represent the following 2D transformations in homogeneous co-ordinate system.

- i) translation ii) rotation
- iii) scaling iv) reflection

June 2012/12 marks

16. What is concatenation transformation? Explain rotation about a fixed point.

June 2012/8 marks, July 2013/8 marks

17. What is transformation? Explain affine transformation.

July 2013/12 marks

Crossword puzzle

Unit 4

Crossword puzzle

Unit 5